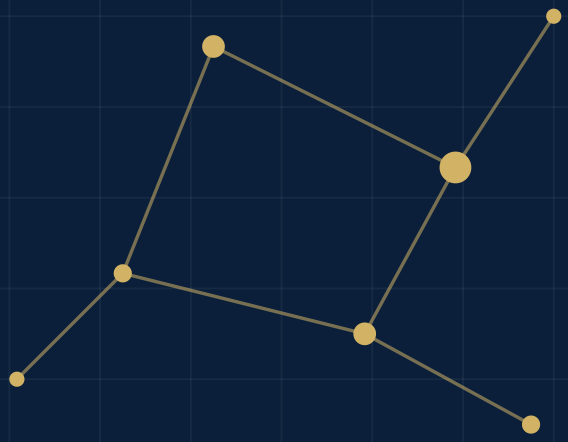
SRM UNIVERSITY · AGENTIC AI & FULL-STACK WORKSHOP



07 — DAY 2 · HACKATHON

Hackathon Prompt Handbook

Hour by hour, prompt by prompt, every term explained

A beginner-friendly plan for the hackathon day: the timeline and checkpoints, every tech word in plain English, ready prompts with blanks for your problem, and a judge-ready checklist.

TIME	PROMPTS	SUBMIT
10:00 to 4:00	H1 to H12	Google Form by 4:00

PREPARED BY

Sathish Kumar K.

AI Educator

simplilearn

Learner handbook · Day 2

November 2026

Contents

Hour by hour, prompt by prompt, every term explained. Follow the sections in order; each one ends with a checkpoint or checklist.

01	About this handbook	3
02	The day at a glance	4
03	Tech words in plain English	5
04	Map of the starter project	8
05	How to write a good prompt	9
06	How your team works	10
07	10:00 – 10:45 · Start and quick setup	11
08	10:45 – 11:00 · Plan with AI	13
09	11:00 – 12:30 · Build the agent	13
10	12:30 – 1:30 · Lunch break	16
11	1:30 – 2:45 · Build more: screens and features	17
12	2:45 – 3:30 · Test, fix and make your slides	18
13	3:30 – 4:00 · Submit in the Google Form	20
14	When things go wrong	21
15	Final checklist	22

01 About this handbook

Today your team builds a working AI agent app for one industry problem, using AI prompts. This handbook is your guide for the whole day: **what to do, when to do it, which prompt to send, and what every technical word means.**

You do not need to know React, Next.js or databases already. Section 3 explains every term in plain English. The prompts do the heavy coding; your job is to give clear instructions, check the result and test it.

Part	What it gives you
Sections 1 to 2	The day's timeline and the 4 checkpoints
Sections 3 to 4	Every tech word explained, and a map of the starter project
Sections 5 to 6	How to write a good prompt, and how your team works together
Sections 7 to 13	The day hour by hour, with a ready prompt for each task (H1 to H12), the 5 slides and the 3:30 to 4:00 submission
Sections 14 to 15	When things go wrong, and the final checklist

Before you start: keep these open: this handbook, your **problem statement PDF** (shared at 10:00 after the lottery) and Guide 6 (Git Step by Step). Yesterday's CampusBite build is your model: today you do the same steps for a new problem.

Remember: every prompt has `<blanks>` in angle brackets. Replace each blank with your own words before you send it.

02 The day at a glance

Time	What happens
10:00 – 10:15	Kickoff, rules, problem lottery
10:15 – 10:30	Read your problem, choose roles, write `AGENTS.md` (H1)
10:30 – 10:45	Quick setup: new repo, your seed data, run (you did this yesterday)
10:45 – 11:00	Plan with AI (H2), show the plan to your mentor
11:00 – 12:30	Build the agent: read tools (H3), first write tool (H4), system prompt (H5)
12:30	**Checkpoint:** one of your own tools works in the chat
12:30 – 1:30	**Lunch break**
1:30 – 2:45	Build more: API route (H6), page (H7), second write tool, extra features
2:45 – 3:30	Developers: test the rules (H8), fix errors (H9), phone check (H10), README (H11). Team Lead and one member: make the 5 slides (H12)
3:30	**Code freeze.** 3:30 is the last time to push code
3:30 – 4:00	**Submit in the Google Form:** 5 slides, GitHub link, Vercel link

> **Key idea:** aim to have a working agent **before lunch**. The afternoon is for screens, testing and polish. A working agent in the chat scores more than a beautiful screen with no agent.

What "done" means

When	Done when
12:30	At least 1 of your own tools runs in the chat with your seed data
3:30	2 or more tools work, 1 writes to Supabase, the Vercel link works, every member has committed
4:00	The Google Form is submitted with your 5 slides, GitHub link and Vercel link

03 Tech words in plain English

You will see these words in prompts, in code and from the judges. Read this section once; come back to it whenever a word is new.

The big picture

Word	What it means	Example
Web app	A program you use in a browser, like Swiggy on the web	Your hackathon project
Frontend	The part the user sees and taps: pages, buttons, lists	The chat and the orders list
Backend	The hidden part that checks rules and talks to the database	The API routes and the agent
Database	Where data is stored safely, even after the app restarts	Your Supabase tables
Full-stack	Building frontend, backend and database together	What your team does today
Client / Server	Client = the user's browser or phone. Server = the computer that runs your backend	Your laptop is the client; Vercel runs the server
localhost	Your own laptop acting as a server, only you can open it	`http://localhost:3000`
Deploy	Put the app on the internet so anyone can open it	Your `.vercel.app` link

Frontend: React, Next.js, TypeScript, Tailwind

Word	What it means	Example
HTML / CSS / JavaScript	Structure / look / behaviour of every web page	A button (HTML), blue (CSS), opens a menu on click (JavaScript)
React	A library for building screens from small reusable pieces	Every order card on a page is the same piece with different data
Component	One reusable piece of screen, written as a function	`OrderCard`, `ChatBox`
Props	The data you pass into a component	`<OrderCard total={240} />`
State	Data inside a component that can change; when it changes, the screen updates	The list of orders after you place a new one
Hook	A special React function that starts with `use`	`useState` keeps state; `useEffect` runs code when the page loads, like fetching data
JSX / TSX	HTML-like code written inside JavaScript or TypeScript files	`return <div>Rs {total}</div>`
Next.js	A framework built on React that gives you pages and APIs in one project	The starter project

Word	What it means	Example
App Router	Next.js rule: folders inside `app/` become web addresses	`app/orders/page.tsx` becomes `/orders`
`page.tsx`	A file that becomes a page you can open	`app/page.tsx` is the home page
`layout.tsx`	The frame around every page (title, fonts)	Rarely changed today
TypeScript	JavaScript with types, so mistakes are caught early	`price: number` cannot hold `"abc"`
Type	A description of the shape of your data	`type Order = { id: number; total: number }`
Tailwind CSS	Style things with short class names instead of CSS files	`bg-green-100 text-green-800 rounded` makes a green badge
Responsive	The page works on phones and laptops	Judges open your link on a phone

Backend: APIs and data

Word	What it means	Example
API	A way for one program to ask another for data or an action; like a waiter	The screen asks `/api/orders` for the order list
API route	A Next.js file that answers API requests	`app/api/orders/route.ts` answers `/api/orders`
`route.ts`	The file name for an API route	Export a `GET` or `POST` function inside it
Request / Response	The question sent to the server / the answer it sends back	Request: "give me my orders". Response: the list
GET / POST	GET reads data; POST sends data to create or change something	GET the menu; POST a new booking
JSON	The text format APIs use for data	`{ "id": 7, "status": "open" }`
Status code	A number that says how the request went	200 ok, 400 bad input, 401 not logged in, 404 not found, 409 conflict, 500 server error
Zod	A library that checks data has the right shape before it is used	Quantity must be a whole number from 1 to 5
Schema	The rules Zod checks against	`z.object({ amount: z.number().positive() })`
Validation	Checking input before using it	Rejecting a negative amount

Database: Supabase

Word	What it means	Example
Supabase	An online PostgreSQL database with a free plan and a browser editor	Your team's project at supabase.com
PostgreSQL	The database system Supabase runs	–
Table / Row / Column	A table holds one kind of thing; a row is one item; a column is one property	`tickets` table, one ticket, its `priority`
Primary key	A unique id for each row	`id`
Foreign key	A column that points to a row in another table	`orders.customer_id` points to `customers.id`
SQL	The language for talking to the database	`SELECT * FROM tickets WHERE status = 'open'`
Query	One request to the database	"All appointments for P001"
Join	Combining rows from two related tables	Order items with their product names
Seed file	A `.sql` file that creates your tables and sample data	`seed/ps4_it_helpdesk.sql`
Database function (RPC)	A function stored in the database that checks rules and changes data in one go	`transfer_funds()` in PS8
RLS (Row Level Security)	Locks the tables so only your server can read them	Already switched on in the seed files

AI and agents

Word	What it means	Example
LLM	Large Language Model: AI that reads and writes text	Gemini
Gemini	Google's AI model that your app uses	`gemini-3.8-flash`
Prompt	The instruction you give an AI	"Show my open tickets"
System prompt	The agent's job description, sent with every chat message	"You are DeskBot, the IT helpdesk assistant..."
Token	A small piece of text; AI usage is counted in tokens	About 4 letters
Agent	AI that can use tools to read and change real data, not just talk	Your app's assistant
Tool	A function the agent is allowed to call	`createTicket`
Read tool	A tool that only looks at data (the agent's eyes)	`getMyOrders`
Write tool	A tool that changes data (the agent's hands)	`bookAppointment`
Function calling	Gemini picks a tool and its arguments; your code runs it	Gemini asks for `searchJobs({ city: "Chennai" })`

Word	What it means	Example
Arguments (args)	The inputs Gemini gives a tool	`{ city: "Chennai" }`
Agent loop	Ask Gemini → run the tool it picks → send the result back → repeat until it answers	Lives in `lib/agent/runAgent.ts`
Business rule	A rule the business must never break	"A slot can be booked only once"

Tools, keys and code

Word	What it means	Example
Terminal	A window where you type commands	`npm run dev`
Node.js / npm	Runs JavaScript on your laptop / installs packages	`npm install`
`package.json`	The list of packages your project needs	–
Environment variable	A setting or secret kept outside the code	`GEMINI_API_KEY`
`.env.local`	The file on your laptop that holds your secrets. Never commit it	5 values
Secret key	A key that can read and change everything; server only	`SUPABASE_SECRET_KEY`
Git / Commit / Push	Version tracking / a saved snapshot / sending snapshots to GitHub	`git commit -m "Add searchJobs tool"`
Repo	Your project on GitHub	`team-07-ps3`
Vercel	The service that puts your app online	Every push to `main` redeploys
Build	Checking and packing the app for the internet	`npm run build`
AI coding agent	Copilot, Antigravity, Codex or Claude Code: writes code from your prompts	Your coding partner today
Diff	The view of what changed: green lines added, red lines removed	Read it before you accept
Rules file	A file the AI coding agent reads with every prompt	`AGENTS.md` or `.github/copilot-instructions.md`

04 Map of the starter project

You start from the same starter as everyone. These are the files you will change. Leave the others alone.

File	What it is	What you do today
`seed/ps<N>_....sql`	Your problem's tables and sample data	Run it in Supabase; read it to learn the table names
`AGENTS.md`	Rules for the AI coding agent	Write it for your problem (H1)
`lib/currentUser.ts`	Who is logged in (you create it)	One line: your problem's user id
`lib/agent/tools.ts`	The agent's tools	Replace the 3 task tools with yours (H3, H4)
`lib/agent/systemPrompt.ts`	The agent's job description	Rewrite it for your problem (H5)
`lib/agent/runAgent.ts`	The agent loop	Usually no change
`app/api/<name>/route.ts`	An API for your screens	Create one for your main table (H6)
`app/page.tsx`	The home page	Change the title and show your data (H7)
`components/`	Screen pieces	Replace `TaskList` with your own list (H7)
`components/AgentChat.tsx`	The chat box	Change the sample prompts (H7)
`README.md`	Your project description	Fill it in for the judges (H11)

How one chat message works (the agent loop)

1. The user types in the chat: "Book me a skin doctor on Friday"
2. `app/api/agent/route.ts` sends the message, the system prompt and your tool list to Gemini
3. Gemini picks a tool and arguments: `searchDoctors({ department: "Dermatology" })`
4. Zod checks the arguments, then your tool reads Supabase
5. The result goes back to Gemini, which may pick another tool (`getAvailableSlots` , then `bookAppointment`)
6. Gemini writes the final answer, and the chat shows every tool it used

> **Key idea:** Gemini never touches your database. It only asks for tools. Your code decides, checks and runs them.

05 How to write a good prompt

Every build prompt today has the same five parts. Yesterday's CampusBite prompts all followed this pattern (Guide 5 explains them line by line).

Part	Question	Example
1. Where	Which file?	"In `lib/agent/tools.ts` ..."
2. What	What to build?	"...add a read tool `getMyTickets` "

Part	Question	Example
3. How	Which pattern to copy?	"...using `defineTool` like the existing examples"
4. Details	Names, arguments, tables, return fields	"reads `tickets` for `CURRENT_USER_ID`, newest first, returns id, title, status, priority"
5. Rules	What must always or never happen?	"Never take a user id as an argument"

Pointing the AI at files

You want to	GitHub Copilot	Google Antigravity
Show the whole project	`#codebase`	`@lib @app @seed`
Show one file	`#file:lib/agent/tools.ts`	`@tools.ts`
Show a terminal error	`#terminalLastCommand`	Select the error, press `Ctrl+L`

The prompts in this handbook use the Copilot form. In Antigravity, swap `#file:` for `@`.

The six golden rules

- One feature per prompt. Small steps are easier to check and fix
- Always name the file and the table. Never let the AI guess names
- Plan in Ask (Copilot) or plan mode (Antigravity); build in Agent or default mode
- Read the diff before you accept. Check the 2 or 3 things that matter
- Test in the app after every prompt
- Commit and push after every step that works

> **Remember:** a prompt with blanks left in `<angle brackets>` gives broken code. Fill every blank.

06 How your team works

Roles (from Guide 3). Teams of 3 or 4 combine roles.

Role	Owns today	Main prompts
Team Lead	Plan, repo, Vercel, README, 5 slides, form	H1, H2, H11, H12
AI / Agent	Tools and system prompt	H3, H4, H5
Backend / API	API routes, Zod, business rules	H4, H6
Frontend	Page and components	H7, H10
Database	Supabase, seed data, checking tables	H8

Driver, reviewer, tester. Only one person types prompts at a time (the driver). One person reads the diff (the reviewer). One person tests in the app (the tester). Swap the driver every hour so everyone commits and everyone's free AI quota is shared.

Avoid clashes. Two people editing the same file at the same time causes Git conflicts. Split work by file: the AI member owns `lib/agent/`, the frontend member owns `app/page.tsx` and `components/`.

Always pull before you start:

```
TERMINAL
git pull
```

Always save after a step works:

```
TERMINAL
git add .
git commit -m "<what you added>"
git push
```

07 10:00 – 10:45 · Start and quick setup

10:00 – 10:15: Kickoff. Listen to the rules. Note your problem number from the lottery.

10:15 – 10:30: Read and assign. Read your problem statement PDF: the business user, logged-in user id, tables, business rules and demo prompts. Choose roles. Then the Team Lead creates the repo (setup step 1 below), and the AI member writes the rules file with H1.

Quick setup (10:30 – 10:45). You did all of this yesterday for CampusBite, so it should take 15 minutes. The only new parts are a new repo and your problem's seed file. You can reuse yesterday's Gemini key.

1. Team Lead: GitHub → starter template → **Use this template** → name `team-<number>-ps<number>` → add teammates as collaborators
2. Everyone: `git clone <repo link>` in `C:\Projects`, then `cd` into it and `npm install`
3. Database member: new Supabase project → **SQL Editor** → run `seed/00_starter_example.sql`, then your `seed/ps<N>_....sql`
4. AI member: use one Gemini key for the team (yesterday's key works)
5. Everyone: copy `.env.example` to `.env.local`, fill in the 5 values, `npm run dev`
6. Team Lead: import the repo on Vercel, add the 5 variables, **Deploy**

H1 · Write your rules file

MODE | Agent (Copilot) or default (Antigravity)

COPILOT PROMPT

```
Read #file:seed/<your seed file>.sql and create AGENTS.md in the project root (and copy the same text to .github/copilot-instructions.md) for our hackathon project.
Problem: <PS number and name>. Business user: <who uses the app>. Logged-in user id: <id from your problem statement, or "none">.
Include, as short bullet points:
- The stack: Next.js App Router, TypeScript, Tailwind, Supabase (server only, secret key), Gemini function calling, Zod.
- The exact table names and their main columns from the seed file.
- Agent tools live in lib/agent/tools.ts and use defineTool() with a Zod schema; descriptions start with "Use this when...".
- The logged-in user id comes from lib/currentUser.ts. Never take a user id as a tool argument.
- Our business rules: <business rules from your problem statement>.
- Never use the Supabase secret key in components. Never commit .env.local.
Keep it under 40 lines.
```

Why: the AI coding agent reads this file with every prompt, so every later answer uses the right table names and rules.

> **Check:** the table names in `AGENTS.md` match your seed file exactly.

> **Save your work (Git):** `git add .` then `git commit -m "Add AGENTS.md rules file"` then `git push`.

Ready to build when: the app runs on `localhost:3000` and your seed tables are in Supabase. Deploy to Vercel now too; it saves a rush at 3:30.

08 10:45 – 11:00 · Plan with AI

H2 · Build plan

MODE | Ask (Copilot) or plan (Antigravity)

COPILOT PROMPT

```
#codebase Read AGENTS.md, this starter and seed/<your seed file>.sql.
Our problem: <business user> has this problem: <problem in one line>.
Our agent must: <what it should do, in 2 lines>.
Give a short build plan:
1. Read tools and write tools (name, arguments, table, what each returns).
2. The business rules each write tool must check before changing data.
3. One API route and the page sections we need.
4. The order to build them in.
Do not write code yet.
```

Check the plan:

- Every table and column name exists in your seed file
- At least 2 tools, and at least 1 writes to Supabase (the agent rule)
- No tool takes the user id as an argument
- The business rules from your problem statement appear in the plan

Show your mentor. If the plan is too big, keep 3 or 4 tools only. A small agent that works beats a big one that does not.

Git: no commit, because no files change. `git status` should say `working tree clean`.

09 11:00 – 12:30 · Build the agent

This is the most important hour. Work in this order: user id, read tools, one write tool, system prompt. Test after each prompt.

Step A: who is logged in

COPILOT PROMPT

```
Create lib/currentUser.ts with: export const CURRENT_USER_ID = "<id from your problem
statement>";
```

Skip this if your problem statement says there is no logged-in user id.

H3 · Read tools (the agent's eyes)

MODE | Agent or default

COPILOT PROMPT

```
In #file:lib/agent/tools.ts replace the 3 task example tools with these read tools for our problem, using defineTool like the existing examples.
- <toolName1>: <args, or "no args">. Reads <table> <where ...>. Order by <column>. Returns <fields>.
- <toolName2>: <args>. Reads <table> joined with <other table> to show <names instead of ids>. Returns <fields>.
Import CURRENT_USER_ID from lib/currentUser.ts and use it inside the tools. Never take a user id as an argument.
Descriptions must start with "Use this when...". Return at most 20 rows. Throw new Error(error.message) on a Supabase error.
```

Example (PS4 DeskBot): `searchKnowledgeBase: arg query (string). Reads kb_articles where title or keywords contain the query (case-insensitive). Returns id, title, category, solution_steps.` and `getMyTickets: no args. Reads tickets for CURRENT_USER_ID, newest first. Returns id, title, status, priority, created_at.`

Check before you accept: the table and column names match the seed file; no tool takes a user id; the old task tools are gone from the `tools` array.

> **Test in the app:** ask a question that needs each tool. Click the grey tool box in the chat to see the arguments Gemini chose.

> **Save your work (Git):** `git add .` then `git commit -m "Add read tools"` then `git push`.

H4 · First write tool (the agent's hands)

MODE | Agent or default

Your write tool must **check the business rules before it changes data**. There are two ways, depending on your problem.

PS1 to PS6: check the rules in TypeScript inside the tool

COPILOT PROMPT

```
In #file:lib/agent/tools.ts add a write tool <toolName> (writes: true).
Args: <arg1 (type, limits)>, <arg2 (type, limits)>. Add Zod limits for each (for example
.min(1).max(5)).
Before changing data, check these rules by reading Supabase, and throw new Error("<clear message
for the user>") if a rule fails:
1. <rule 1, for example: the slot exists and is_booked is false>
2. <rule 2, for example: the slot is in the future>
If all rules pass: <insert into table ... / update table ... set ...> for CURRENT_USER_ID, and
return <fields>.
Description: "Use this when <the user wants to ...>". Add it to the tools array.
```

PS7 and PS8: call the database function (like CampusBite)

COPILOT PROMPT

```
In #file:lib/agent/tools.ts add a write tool <toolName> (writes: true).
Args: <args with types and limits>. Call getSupabase().rpc("<function name>", { <p_param>:
<value>, ... }).
Throw new Error(error.message) on error, else return data.
Do not update the tables directly in TypeScript; the database function checks all the rules.
Description: "Use this when <...>". Add it to the tools array.
```

The function names and parameters are at the end of your seed file, in the lines that start with `create or replace function`.

Check before you accept: every rule from your prompt is in the code as an `if` check that throws an error; the write uses `CURRENT_USER_ID`; `writes: true` is set.

> **Test in the app:** one request that should work, and one that breaks a rule. The agent should refuse with your error message.

> **Save your work (Git):** `git add .` then `git commit -m "Add <toolName> write tool"` then `git push`.

H5 · System prompt (the agent's job description)

MODE | Agent or default

COPILOT PROMPT

```
Rewrite buildSystemPrompt in #file:lib/agent/systemPrompt.ts for our app:
- You are <AppName>, the <friendly / professional> assistant for <business user>. The logged-in user is CURRENT_USER_ID.
- Keep the line with today's date.
- Always use tools for <the data in our tables>. Never invent data.
- Never guess an id. Call a read tool first to find real ids, even if you listed them earlier.
- Before <write action>, check <what to check> with the read tools.
- If a rule stops an action, explain why in one sentence and suggest what the user can do instead.
- Only call <write tool> when the user clearly asks for it. Never undo an action to fix your own mistake; explain instead.
- After a change, say exactly what changed: <fields to show, for example ticket id, priority and team>.
- <Safety rule from your problem statement, if any>.
- Keep replies short. Only help with <our topic>.
```

Then, if your agent needs several tools for one request:

COPILOT PROMPT

```
In #file:lib/agent/runAgent.ts change MAX_ROUNDS from 5 to 8.
```

Test in the app: refresh the page, then try the main demo prompt from your problem statement. Then try an unclear request ("do the usual") and an off-topic one ("write me a poem").

Save your work (Git): `git add .` then `git commit -m "Add system prompt for <AppName>"` then `git push`.

Checkpoint: by 12:30, at least one of your own tools works in the chat. If not, tell your mentor before lunch.

10 12:30 – 1:30 · Lunch break

Before you leave:

- [ ] Push everything (`git status` says `working tree clean`)
- [ ] Write down what works and what is next, in 3 lines, in your team chat

- [ ] Check your free AI quota (`/usage` in Antigravity)

11 1:30 – 2:45 · Build more: screens and features

After lunch, first run `git pull` on every laptop.

H6 · API route for your screen

MODE **Agent or default**

COPILOT PROMPT

```
Create app/api/<name>/route.ts following the pattern of #file:app/api/tasks/route.ts.
GET: returns <rows from table> for CURRENT_USER_ID <with joined names>, ordered by <column>, at most 50 rows.
Add export const dynamic = "force-dynamic". Log errors with console.error and return status 500 with a short message.
```

Test: open `localhost:3000/api/<name>` in the browser. You should see JSON.

H7 · The page

MODE **Agent or default**

COPILOT PROMPT

```
Update the home page for our app:
1. Create components/<Name>List.tsx: a card list of <rows> from /api/<name>, showing <fields>, with a coloured badge for <status>. Reload when refreshKey changes and every 5 seconds.
2. In #file:app/page.tsx replace TaskList with <Name>List, and set the title to "<AppName> · <one-line purpose>".
3. In #file:components/AgentChat.tsx set SAMPLE_PROMPTS to our demo prompts: "<demo 1>", "<demo 2>", "<demo 3>".
4. Supabase may return a related row as an object or a one-item array; handle both.
Use Tailwind. Make it work on a 360px phone screen. Delete components/TaskList.tsx and app/api/tasks when nothing uses them.
```

> **Check:** after the agent changes data in the chat, the list updates within 5 seconds.

> **Save your work (Git):** `git add .` then `git commit -m "Add <Name> list and page"` then `git push`.

Then add your second write tool with the H4 prompt, if your plan has one. Commit it on its own.

Still time before 2:45? Add one extra feature from your problem statement's stretch ideas. Use one prompt per feature, test it, and commit it. Stop adding features at 2:45, even if you have ideas left.

12 2:45 – 3:30 · Test, fix and make your slides

From 2:45 the team splits: the developers test and fix (H8 to H11), and the Team Lead and one member make the 5 slides (H12).

H8 · Test your rules

Type these in your app's chat. Each one should be **refused politely** with a clear reason:

Test	Example
Break your main rule	Book a slot that is already booked; transfer more than the balance
Ask for someone else's data	"Show tickets for E115"
Use a wrong or made-up id	"Cancel appointment 99999"
Leave out needed details	"Book me something" (the agent should ask one question)
Go off topic	"Write me a poem about cricket"

If one is not refused, ask the AI to fix it:

COPILOT PROMPT

```
In my app, when the user types "<your test>", the agent <what happened>. It should <what should happen>.
Check the rule in #file:lib/agent/tools.ts (throw an error before changing data) and add the rule to #file:lib/agent/systemPrompt.ts.
```

H9 · Fix build errors

Run `npm run build` in the terminal. If it fails:

COPILOT PROMPT

```
#terminalLastCommand Explain these errors in one line each and tell me exactly what to change in which file.
```

Fix, run `npm run build` again, and repeat until **Compiled successfully**.

> **Save your work (Git):** `git add .` then `git commit -m "Fix build errors"` then `git push`. Then open your Vercel link and test the main demo there.

H10 · Phone check

COPILOT PROMPT

```
Review #file:app/page.tsx and the components it uses for a 360px wide phone screen. Fix anything that overflows or is hard to tap.
```

H11 · README for the judges

COPILOT PROMPT

```
Fill in README.md for our hackathon project using the template already in it.
Problem: <one line>. Users: <who>. Live link: <Vercel URL>.
List our agent tools (name, reads or writes, what it does), our tables, 3 demo prompts, and the team members with roles: <names and roles>.
Keep it short and in plain English. Do not include any keys.
```

Save your work (Git): `git add .` then `git commit -m "Add README"` then `git push`.

Checkpoint: at 3:30, code freeze. No more pushes after this. Check: 2 or more tools work, 1 writes to Supabase, the Vercel link works, and every member has at least one commit (`git log` on GitHub).

H12 · Your 5 slides

Make the slides in Google Slides or PowerPoint while the developers test. Use this prompt in ChatGPT or Gemini to draft the text, then check every line is true for **your** app.

COPILOT PROMPT

```
Write the text for 5 presentation slides for our hackathon project. Plain English, at most 5 short bullet points per slide.
Project: <AppName>, problem <PS number>. Team: <names and roles>.
Problem: <business user> has this problem: <one line>.
What our app does: <2 lines>. Our agent tools: <tool names, and which ones read or write>.
Business rules our app checks: <rules>. Demo prompt that works: "<prompt>". A request it refuses: "<prompt>".
Links: GitHub <repo link>, live app <Vercel link>.
Slides: 1 Title and team, 2 The problem, 3 Our solution and demo, 4 How the agent works, 5 Results and links.
```

What goes on each slide

Slide	Title	What to show
1	Title and team	App name, problem number and name, team number, members and roles

Slide	Title	What to show
2	The problem	Who the user is, their problem today, why it matters (1 or 2 numbers from your seed data help)
3	Our solution and demo	2 screenshots: the chat with the tools the agent used, and the data on screen after it changed. Write the demo prompt under the screenshot
4	How the agent works	The flow: user → agent → tools → Supabase. List your read tools and write tools, the business rules you check, and the stack (Next.js, Supabase, Gemini, Zod, Vercel)
5	Results and links	What works, a rule the agent refuses, what you would add next, your GitHub link and Vercel link

Tips for good slides

- One idea per slide, large text, no long paragraphs
- Screenshots of **your** app beat any stock picture. Take them before 3:30 (`Win+Shift+S` on Windows)
- Never put keys, passwords or the Supabase dashboard in a screenshot
- Save as PDF if the form asks for a file, or share the Google Slides link with **Anyone with the link can view**

Questions judges often ask, and how to answer

Question	A good answer covers
"How does the agent decide what to do?"	Gemini reads the tool descriptions and picks a tool; our code runs it
"What stops the AI from breaking your rules?"	The write tool checks the rule in code (or the database function) before changing data
"Why don't you pass the user id to the tool?"	So the user cannot act as someone else; the server decides who is logged in
"What does Zod do here?"	Checks the arguments Gemini sends before the tool runs
"Where are your keys?"	In `.env.local` and Vercel settings only, never in GitHub
"Who wrote this part?"	The member who owns it explains it, even if AI helped write it

> **Checkpoint:** at 3:30, code freeze. No more pushes after this. Check: 2 or more tools work, 1 writes to Supabase, the Vercel link works, every member has at least one commit, and the 5 slides are ready.

13 3:30 – 4:00 · Submit in the Google Form

From 3:30 to 4:00, the Team Lead fills in the **Google Form** shared by the trainer. You submit three things, plus your team details:

Submit	How to get it ready
5 slides	PDF file, or a Google Slides link set to **Anyone with the link can view**
GitHub link	The address of your repo page, for example `github.com/<account>/team-07-ps3`. The repo must be **Public**
Vercel link	Vercel → your project → **Domains**, for example `team-07-ps3.vercel.app`
Team details	Team number, problem number, members' names and roles
App passcode	From `.env.local` (`APP_PASSCODE`), so judges can log in to your app

Before you submit, check:

- [ ] The GitHub link opens in a private (incognito) browser window
- [ ] The Vercel link opens on a phone and the passcode works
- [ ] The main demo prompt works on the Vercel link, not only on localhost
- [ ] The slides link opens for someone outside your team

> **Remember:** submit once, carefully, before 4:00. A late form may not be counted.

14 When things go wrong

Problem	Do this
The AI used table or column names that do not exist	"Use only the tables and columns in #file:seed/<file>.sql. Fix the names in #file:lib/agent/tools.ts."
Agent does not call your tool	Check the tool is in the `tools` array and its description starts with "Use this when..." and names the user's words
Agent guesses ids or loops ("too many steps")	Add the "never guess an id" rule to the system prompt; raise `MAX_ROUNDS` to 8
`Could not find the table ... in the schema cache`	`.env.local` points to another Supabase project, or the seed was not run there
Gemini `429`	Too many requests. Wait one minute; send shorter prompts
A prompt broke the app	`git restore .` takes you back to your last commit
Vercel works locally but not online	A variable is missing on Vercel; add it and **Redeploy**
`git push` rejected	A teammate pushed first: `git pull --rebase`, then `git push`
Free AI quota running low	Let another teammate drive; use Tab completions for small edits

Problem	Do this
Stuck for 10 minutes	Raise your hand. Show your mentor the exact error and the step

> **Remember:** if time is short, cut features, not testing. Two tools that work in the demo are worth more than five that crash.

15 Final checklist

Agent (30 marks)

- [ ] 2 or more tools, at least 1 writes to Supabase
- [ ] Gemini picks the tools itself; no `if` on the user's words
- [ ] Zod checks every tool's arguments
- [ ] The agent refuses a request that breaks a rule, with a clear reason

Working app (25 marks)

- [ ] The main demo works on the Vercel link, not only on localhost
- [ ] The data change shows on screen

Code and GitHub (15 marks)

- [ ] Every member has commits
- [ ] README is filled in
- [ ] No keys in the repo; `.env.local` is not on GitHub

UI and UX (10 marks)

- [ ] Works on a phone screen
- [ ] The user can see which tools the agent used

Slides and explanation (20 marks)

- [ ] 5 slides ready, with screenshots of your own app
- [ ] Every member can explain their own part

Submission (by 4:00)

- [ ] Google Form submitted with the 5 slides, GitHub link and Vercel link